

Distributed Authentication and Authorization Vulnerability Analysis: An Empirical Evaluation of Keycloak and FastAPI Implementations

Pooria Madani Sani
Department of Computer Engineering
Ferdowsi University of Mashhad
Network Security Advanced Laboratory Report

May 2026

Abstract

This technical report presents a distributed authentication and authorization security assessment focused on modern assertion-based protocols and their operational hardening. The study situates itself within the architectural shift from implicit credential sharing toward explicit delegation and claims-based identity exchange, as formalized in OAuth 2.0 and OpenID Connect, and emphasizes the heightened exposure surfaces of decentralized network architectures that depend on token issuance and validation across trust boundaries. An empirical testbed is constructed using a self-hosted Keycloak Identity Provider (IdP) coupled with a FastAPI service that represents a resource server in a multi-service environment. The evaluation intentionally explores structural cryptographic weaknesses, including the `alg: none` signature bypass vector and related token integrity failures, to quantify the impact of misconfiguration and weak validation logic on authorization outcomes. Defensive remediations are engineered and verified through a layered control strategy that includes Proof Key for Code Exchange (PKCE), Refresh Token Rotation (RTR), and stringent token lifespan constriction aligned with operational risk. The effectiveness of these countermeasures is validated programmatically through rigorous automated testing using `pytest`, ensuring that both negative and positive security assertions are enforced under repeatable conditions. The resulting analysis offers a precise, evidence-driven characterization of failure modes and mitigations for distributed identity systems.

Contents

Abstract	i
1 Introduction and Scope	1
1.1 Document Overview and Research Motivation	1
1.2 System Security Topology and Objectives	1
1.3 Methodology and Comprehensive 5-Phase Roadmap	2
2 Theoretical Foundations of Modern Token-Based Authentication and Authorization Protocols	3
2.1 Architectural Breakdown of the Four Core OAuth 2.0 Actors (RFC 6749)	3
2.2 Cryptographic Token Taxonomy	4
2.3 Anatomy and Cryptographic Deconstruction of JSON Web Tokens (JWT)	4
2.4 Cryptographic Signing Schemes: Symmetric vs. Asymmetric Paradigms	5
2.5 Tooling and Documentation Foundations	5
3 Laboratory Testbed Design and Environment Isolation	7
3.1 Identity Provider (IdP) Selection Evaluation: Keycloak vs. Cloud-Based IdPs	7
3.2 Isolated Network Architecture and Component Topology	7
3.3 Comprehensive Containerized Component Specifications	8
3.4 Diagnostic Utilities and Analytical Tooling	9
4 Scenario Implementation, Vulnerability Realization, and Attack Simulation	10
4.1 Intentional Security Degradation of the Identity Provider	10
4.2 Baseline Authentication Flow Execution	10
4.3 Mechanics of the Signature Bypass Attack (<code>alg: none</code>)	11
4.4 Exploitation Phase and Vulnerable Backend Analysis	11
5 Defensive Strategies and Infrastructure Hardening	14
5.1 Architectural Shift: Mitigating Interception via PKCE	14
5.2 The Hardened Resource Server Implementation	14
5.3 Session Lifespan Constriction and Refresh Token Rotation (RTR)	15
5.4 Mitigation Proof: Exploit Rejection and Validation	16
6 Automated Integration and Empirical Verification Test Results	17
6.1 Automated Security Testing Methodology	17
6.2 Empirical Test Execution and Log Analysis	17
6.3 Significance of Security Regressions in DevSecOps	18
7 Conclusion and Security Recommendations	19
7.1 Conclusion and Key Findings	19
7.2 Security Recommendations for Cloud-Native Architectures	19

Chapter 1

Introduction and Scope

1.1 Document Overview and Research Motivation

Distributed API ecosystems have become the dominant substrate for enterprise and academic software systems, yet their trust models are frequently assembled from heterogeneous services with divergent security controls. In this environment, the security of authentication and authorization flows is not only a local application concern but a systemic requirement that must hold across a network of endpoints, intermediaries, and identity providers. Legacy password-sharing schemes couple credential storage and validation to application code, creating brittle perimeter assumptions and a high-value compromise target. Modern authorization frameworks, particularly those standardized in RFC 6749, replace direct credential propagation with delegated token exchange, where authorization decisions are derived from verifiable claims and scope-bound access tokens. This transition introduces new cryptographic and architectural challenges, including robust token verification, cross-service trust alignment, and the prevention of downgrade or bypass vectors. The five-phase methodology that guides this report is detailed in Section 1.3 and establishes a rigorous empirical baseline for evaluating these risks.

This report is deliberately synthesized from reviewed scholarly articles, technical reports, and standards documentation, and it is structured as a technical narrative that distills those sources into a reproducible security analysis. The bibliography at the end of the report provides direct local links to every referenced artifact in the References folder, preserving traceability between the analytical claims and their originating documents.

1.2 System Security Topology and Objectives

The central objective of this laboratory research project is to assess the resilience of a distributed authentication and authorization topology when identity validation is externalized from application logic into a dedicated Identity Provider. This separation of concerns is fundamental to modern security architecture, where the IdP acts as a specialized authority responsible for credential verification, token issuance, and policy-aware authorization metadata. The testbed is intentionally designed to surface boundary conditions between the IdP, the FastAPI resource server, and the client-side authorization code flow, with a focus on how perimeter definitions evolve when security logic is distributed across trust domains. By evaluating how authorization is enforced at the service boundary, the project seeks to clarify the operational consequences of misconfiguration, weak validation, and insufficient token hygiene, while also demonstrating how a properly configured IdP reduces exposure by enforcing consistent identity validation outside of application code.

1.3 Methodology and Comprehensive 5-Phase Roadmap

The research trajectory is organized into a five-phase roadmap that aligns theoretical rigor with operational realism, and each phase is constructed to feed dependencies into the next. Phase 1 establishes the theoretical foundations and mathematical analysis required to interpret token lifecycles, signature verification semantics, and protocol invariants, which is essential for framing later attack hypotheses. Phase 2 operationalizes the laboratory testbed, emphasizing environment isolation, threat surface control, and reproducible deployment of the Keycloak IdP and FastAPI resource server so that later observations remain attributable and not confounded by infrastructure variability. Phase 3 then realizes vulnerabilities through controlled attack simulation, where flawed cryptographic assumptions and relaxed verification logic are exploited to observe authorization deviations under realistic client behaviors.

Phase 4 applies defensive engineering and infrastructure hardening, translating the findings from attack simulation into concrete mitigations such as PKCE enforcement, refresh token rotation, and reduced token lifetimes that tighten the temporal and cryptographic bounds of access. Phase 5 completes the empirical loop by automating validation and evidence gathering with systematic testing, where exploit attempts and remediation checks are encoded into repeatable `pytest` suites to ensure that security properties are not regressions but enforced invariants. Together, these phases form a coherent methodology that couples protocol theory with implementation evidence while preserving a strict causal chain from design assumptions to measured security outcomes.

Chapter 2

Theoretical Foundations of Modern Token-Based Authentication and Authorization Protocols

2.1 Architectural Breakdown of the Four Core OAuth 2.0 Actors (RFC 6749)

OAuth 2.0 formalizes a distributed authorization architecture through four distinct actors whose interactions are defined by explicit trust boundaries and carefully scoped responsibilities. The Resource Owner represents the principal who ultimately controls the protected assets, such as personal data or delegated privileges. This actor is the source of consent, but in a secure deployment the Resource Owner does not directly expose credentials to downstream services. Instead, consent is expressed through an authorization grant that binds scope, audience, and temporal constraints. The Client Application is the entity requesting access on behalf of the Resource Owner, and it operates within a trust boundary that is separate from the Resource Server. The client is permitted to request tokens, but it is not a validator of those tokens by default; its function is to initiate authorization, present proof of possession or authorization code, and handle the token exchange flow. The Authorization Server, often realized as an Identity Provider, is the authoritative issuer of tokens and the only actor entrusted with authenticating the Resource Owner. Its trust domain is defined by its ability to generate cryptographically bound assertions and enforce policies such as consent, scope filtering, and session state. Finally, the Resource Server hosts protected resources and validates incoming access tokens to enforce authorization. Its trust boundary is orthogonal to the Authorization Server, and it must apply strict verification of token integrity, issuer, audience, and temporal validity.

This separation of responsibilities contrasts sharply with the legacy password-sharing anti-pattern in which the client or downstream service directly receives and handles Resource Owner credentials. In that model, the blast radius of compromise is systemic: any party holding the password becomes an implicit superuser, and credential leakage compromises all services that reuse or trust the same credential. OAuth 2.0 and related assertion-based frameworks replace direct credential propagation with token delegation, where the Authorization Server issues signed assertions with scoped privileges and explicit lifetimes. The decoupled architecture shifts token validation perimeters away from localized codebases by introducing a dedicated trust anchor in the Authorization Server and by imposing verification duties on Resource Servers rather than on every client or intermediary. As a result, authorization decisions become a function of cryptographically verifiable claims rather than shared secrets, enabling distributed systems to enforce uniform policy while reducing the exposure of primary credentials.

2.2 Cryptographic Token Taxonomy

Token-based security architectures typically differentiate between access tokens and identity tokens because they serve distinct authorization and authentication goals. Access tokens embody delegated authorization and are presented to Resource Servers to obtain protected resources, while identity tokens encapsulate assertions about authentication events and subject identity for consumption by clients. This distinction is not semantic alone; it determines the intended consumer, the verification path, and the way claims are interpreted within a distributed security boundary. A rigorous comparison clarifies the operational role of each token class and the specification families that govern their issuance and validation.

Table 2.1: Access Token and ID Token Comparison Matrix

Dimension	Access Token	ID Token
Core Intent/Purpose	Delegated authorization to access protected resources and APIs under scoped privileges.	Authentication assertion describing the subject and the authentication event.
Primary Consumer Node	Resource Server or API gateway enforcing authorization.	Client Application validating user identity and session context.
Cryptographic Structure/Format	Frequently opaque, but may be structured JWT for self-contained validation.	Structured JWT with standardized claims for interoperability.
Profiles and Specifications	OAuth 2.0 (RFC 6749) and profile-specific constraints.	OpenID Connect Core with claim semantics and authentication context.
Typical Lifespan Constraints	Short-lived, often minutes, to reduce exposure from token theft.	Short-lived but aligned with session policies and re-authentication requirements.

The access token is therefore a minimal, resource-centric bearer or proof-of-possession artifact, whereas the ID token is a subject-centric assertion that binds authentication context to an audience. In distributed systems, confusing these roles leads to authorization flaws such as accepting ID tokens at resource boundaries, or using access tokens to infer identity without verifying audience and issuer constraints. The operational separation also encourages targeted validation paths: access tokens are validated by resource servers against expected scopes and audiences, while ID tokens are validated by clients to establish user identity and session integrity.

2.3 Anatomy and Cryptographic Deconstruction of JSON Web Tokens (JWT)

JSON Web Tokens encode security assertions in a compact, URL-safe format that is designed to be transported across heterogeneous systems. A JWT is composed of three Base64URL-encoded segments: the header, the payload, and the signature. The header declares the token metadata, including the signing algorithm (for example, `alg` set to `HS256` or `RS256`) and an optional key identifier (`kid`) to support key rotation. The payload carries the claims, including registered claims such as issuer (`iss`), subject (`sub`), audience (`aud`), and temporal bounds (`iat`, `exp`). These claims are not confidential by default and must be treated as integrity-protected but publicly readable data unless explicitly encrypted via JWE. The signature binds the header and payload to an issuing authority and enables integrity verification across distributed resource servers.

Base64URL encoding is a transport encoding, not a cryptographic protection mechanism.

Formally, it encodes binary input into an ASCII-safe representation by substituting URL-safe characters and typically omitting padding. If $\text{b64url}(x)$ denotes the Base64URL encoding of input x , then the signing input is the concatenation of the encoded header and payload, separated by a period. The JWT structure can be expressed as:

$$\text{JWT} = \text{b64url}(\text{Header})\|".\|\text{b64url}(\text{Payload})\|".\|\text{b64url}(\text{Signature}) \quad (2.1)$$

Encoding does not provide confidentiality, and therefore any sensitive claim placed in the payload becomes observable to intermediaries. The security of the JWT thus depends on correct signature verification and contextual validation of claims rather than secrecy.

The signature generation process binds the header and payload to a key and algorithm. Let $S = \text{b64url}(\text{Header})\|".\|\text{b64url}(\text{Payload})$ denote the signing input. For a symmetric scheme such as HS256, the signature is computed using an HMAC keyed by a shared secret K :

$$\text{Sig}_{\text{HS256}} = \text{HMAC}_{\text{SHA256}}(K, S) \quad (2.2)$$

For an asymmetric scheme such as RS256, the issuer signs the hash of S using the private key K_{priv} :

$$\text{Sig}_{\text{RS256}} = \text{RSA_Sign}_{K_{\text{priv}}}(\text{SHA256}(S)) \quad (2.3)$$

Verification recomputes or validates the signature and confirms that S is unmodified, which cryptographically binds the claims to the issuer and prevents tampering when the correct algorithm and key are enforced.

2.4 Cryptographic Signing Schemes: Symmetric vs. Asymmetric Paradigms

HS256 and RS256 represent two fundamentally different security models for token issuance and validation. HS256 uses a shared secret that must be known by both the issuer and any service that validates the token. This simplicity yields efficient signing and verification, but it implies that every validating resource server holds the same secret, which increases the blast radius if any single resource server is compromised. In distributed architectures, the symmetric model can become operationally fragile because secret distribution, rotation, and compartmentalization are challenging to maintain at scale. Any leakage of the shared secret enables adversaries to mint valid tokens, thereby undermining the trust boundary between issuer and resource server.

RS256 implements an asymmetric paradigm in which the Identity Provider alone holds the private signing key K_{priv} , while resource servers validate signatures using the corresponding public key K_{pub} . The public key is typically distributed through a JSON Web Key Set (JWKS) endpoint, enabling automated key rotation and verification without exposing the signing secret. Verification can be formalized as:

$$\text{Verify}_{K_{\text{pub}}}(\text{Sig}_{\text{RS256}}, S) = \text{true} \quad (2.4)$$

This model ensures that a compromise of one resource server does not allow an attacker to forge new tokens, because the attacker never gains access to K_{priv} . As a result, the asymmetric approach significantly limits compromise proliferation in decentralized architectures and enables a strict separation between token issuance authority and token validation nodes, aligning with the principle of least privilege and reducing systemic risk.

2.5 Tooling and Documentation Foundations

The theoretical foundation of this project includes an explicit mapping between protocol semantics and the capabilities of the tooling used to validate them. Keycloak is treated as the

authoritative IdP for realm configuration, token issuance, and JWKS publication, enabling controlled evaluation of OAuth 2.0 and OIDC parameters under both weak and hardened policies. FastAPI and PyJWT provide the application-level enforcement surface for token verification, while `pytest` supplies the formal automation layer needed to express security requirements as repeatable assertions. Docker Compose provides deterministic service topology and network isolation, and diagnostic tooling such as JWT.io, Postman, and cURL supports controlled inspection and adversarial request crafting at the protocol boundary.

Project documentation and external standards are treated as primary theoretical references rather than ancillary materials. The standards corpus (for example, RFCs for OAuth 2.0, JWT, PKCE, and related extensions) and vendor documentation are used to define invariant protocol behavior, claim semantics, and algorithm constraints. This theoretical framing guides the experimental design in later chapters and ensures that implementation choices and security validations are grounded in authoritative source material rather than ad hoc assumptions.

Chapter 3

Laboratory Testbed Design and Environment Isolation

3.1 Identity Provider (IdP) Selection Evaluation: Keycloak vs. Cloud-Based IdPs

The selection of a self-hosted Keycloak Identity Provider is a deliberate methodological choice driven by the security objectives of this laboratory. Managed cloud IdPs such as Auth0 and Okta provide robust, production-grade controls, but their strength is precisely what makes them unsuitable for adversarial experimentation. A security evaluation that aims to discover and validate weak configurations must operate under full administrative autonomy, including access to realm-level policies, token issuance templates, signature verification parameters, and protocol toggles that are often hidden or immutable in managed environments. Keycloak allows the research process to expose the identity core as a controllable system rather than a black-box service, enabling the measurement of both failure modes and remediation effects with strong causal attribution.

Self-hosting is particularly important for testing the boundaries of protocol correctness. Cloud IdPs are designed to prevent unsafe settings, which is operationally desirable in production but analytically restrictive in a laboratory environment where the objective is to observe misconfiguration impact. In this study, the ability to enable legacy or deprecated flows, intentionally relax token validation, adjust lifetimes beyond recommended values, or alter signature header behavior is essential for reproducing known vulnerability classes and quantifying their operational blast radius. Keycloak supports explicit configuration of token lifespans, client capabilities, and algorithm constraints at the realm and client levels, which permits controlled weakening of cryptographic policy for empirical analysis. This level of precision cannot be guaranteed with managed IdPs, where provider-side mitigations may silently override or constrain unsafe settings, obscuring the conditions required for meaningful security experimentation.

3.2 Isolated Network Architecture and Component Topology

The laboratory testbed is constructed as a multi-tier distributed architecture that mirrors a segmented enterprise environment while remaining fully reproducible. The topology is organized around three primary nodes: a client tier responsible for initiating authorization flows, a central Identity Provider that acts as the authorization authority, and a decoupled FastAPI Resource Server that enforces access controls at the service boundary. Each node is encapsulated within a containerized runtime to provide deterministic configuration, controlled dependencies, and clean reset capability between experiments. This approach creates a well-defined trust boundary around each node and allows for precise observation of how token validation behavior changes across tiers.

Network isolation is achieved by deploying the components onto a dedicated Docker network that emulates a private service plane. The client tier interacts with the Identity Provider via standard OAuth 2.0 and OpenID Connect endpoints, while the Resource Server accepts bearer tokens over stateless HTTP/REST interfaces. The design ensures that no implicit trust is granted across tiers: the client does not validate tokens, the Identity Provider does not access protected resources, and the Resource Server does not authenticate users directly. This separation enables a clear mapping between protocol behavior and enforcement decisions, as each request traverses a single, verifiable chain of responsibility from token issuance to resource access.

3.3 Comprehensive Containerized Component Specifications

The containerized deployment aligns service configuration with the intended security model and provides a stable environment for repeated experimentation. The Identity Provider node runs the official Keycloak container image and exposes port 8080 for administrative access and protocol endpoints. Realm state, cryptographic material, and client settings are persisted through mounted volumes to ensure consistent behavior across restarts, while environment variables define administrative credentials and bootstrap configuration. The client node is implemented as a Python-driven application tier that exercises both front-channel and back-channel interactions, enabling direct control over authorization requests, token exchange, and proof mechanisms. The Resource Server node runs a lightweight FastAPI application that validates inbound bearer tokens, checks issuer and audience claims, and authorizes access to high-privilege routes only after cryptographic verification passes.

The following configuration illustrates the structured service topology, shared network, and environment definitions used to emulate a production-grade security boundary in a controlled lab setting.

```
1 version: "3.9"
2
3 services:
4   keycloak:
5     image: quay.io/keycloak/keycloak:24.0
6     command: ["start-dev", "--http-port=8080"]
7     environment:
8       KEYCLOAK_ADMIN: admin
9       KEYCLOAK_ADMIN_PASSWORD: admin
10    ports:
11      - "8080:8080"
12    volumes:
13      - keycloak_data:/opt/keycloak/data
14    networks:
15      - lab_net
16
17   resource_server:
18     build: ./secure_server
19     environment:
20       OIDC_ISSUER: http://keycloak:8080/realms/security-lab
21       OIDC_AUDIENCE: fastapi-resource
22       JWKS_URL: http://keycloak:8080/realms/security-lab/protocol/openid-connect/certs
23     ports:
24       - "8000:8000"
25     depends_on:
26       - keycloak
27     networks:
28       - lab_net
```

```

29
30     client:
31         build: ./scripts
32         environment:
33             OIDC_ISSUER: http://keycloak:8080/realms/security-lab
34             CLIENT_ID: lab-client
35             CLIENT_SECRET: lab-secret
36             REDIRECT_URI: http://localhost:9000/callback
37         depends_on:
38             - keycloak
39             - resource_server
40         networks:
41             - lab_net
42
43 networks:
44     lab_net:
45         driver: bridge
46
47 volumes:
48     keycloak_data:

```

Listing 3.1: Containerized testbed topology for Keycloak, client tier, and FastAPI resource server

This composition ensures that each service is reachable only through the controlled lab network, preserving the integrity of experimental variables such as token lifecycle parameters and verification strategy. By anchoring the Resource Server to a predictable JWKS endpoint and a fixed issuer claim, the testbed enables deterministic validation logic that can be systematically stressed by manipulated tokens or boundary-case claims.

3.4 Diagnostic Utilities and Analytical Tooling

The experimental workflow relies on a set of diagnostic utilities that provide both infrastructure control and protocol visibility. Docker Compose serves as the orchestration backbone, enabling deterministic startup order, consistent network isolation, and rapid teardown between test runs. Its declarative configuration ensures that service versions, environment variables, and network boundaries remain fixed across iterations, which is essential for attributing observed behavior to security variables rather than infrastructure drift. The orchestration layer also facilitates rapid pivoting between vulnerable and hardened configurations by allowing controlled changes to environment variables and service command flags.

JWT.io is used as a structural inspection and manipulation utility for tokens. Its visual deconstruction of the header, payload, and signature provides immediate visibility into cryptographic metadata and claim structure, while controlled tampering of claims and algorithm fields allows for targeted testing of validation logic. Postman and cURL complement this by enabling raw HTTP request crafting outside of browser constraints. They are used to send authorization requests, token exchanges, and resource access attempts with carefully manipulated headers, query parameters, and bearer tokens. This toolchain provides an end-to-end experimental pipeline that couples infrastructure isolation with precise protocol-level inspection and adversarial request generation.

Chapter 4

Scenario Implementation, Vulnerability Realization, and Attack Simulation

4.1 Intentional Security Degradation of the Identity Provider

The vulnerability phase begins with explicit, controlled degradation of the Keycloak realm and client configuration to emulate a legacy deployment where operational constraints or administrative drift have weakened protocol controls. In the vulnerable client configuration, the deprecated implicit flow is deliberately enabled while the standard authorization code flow is disabled, thereby forcing token delivery through the browser front channel. This design choice introduces a predictable exposure pathway for access tokens, as they can appear in URLs or browser storage during redirect processing. In parallel, the access token lifetime is expanded to 36,000 seconds (10 hours), which materially increases the time window in which a leaked or intercepted token remains valid. The configuration also adopts a permissive wildcard redirect URI pattern, specifically `https://jwt.io/*`, which weakens redirect target validation and makes it possible for adversaries to capture tokens by controlling or abusing redirection surfaces.

Each of these modifications broadens the attack surface in a distinct and compounding manner. Enabling the implicit flow removes the confidentiality guarantees associated with back-channel exchanges and makes the token transit path observable within client-side contexts. A 10-hour token lifespan creates a persistent bearer credential that can be reused long after its issuance, undermining temporal containment strategies and complicating detection. The wildcard redirect URI expands the acceptable destination set for tokens, which transforms the redirect validation step into a permissive gate that can be bypassed through crafted URLs. Together, these choices simulate a realistic misconfiguration profile in which protocol-level safeguards are weakened at multiple layers, providing a fertile environment for testing forged token acceptance and boundary enforcement in the resource server.

4.2 Baseline Authentication Flow Execution

Before introducing adversarial manipulation, the nominal authentication flow is executed to confirm that the system behavior is coherent under the intentionally weakened configuration. The client application initiates the authorization request against the Identity Provider, which authenticates the user and issues a token in the front channel according to the implicit flow semantics. This baseline flow validates the operational continuity of the testbed and establishes a ground truth for comparison when the token is later mutated or forged.

Because the implicit flow returns tokens through the browser-facing channel, the resulting JWT is immediately observable and extractable within the client context. This exposure is

not simply a convenience for the attacker; it is a structural vulnerability that undermines the confidentiality assumptions usually protected by back-channel token exchanges. The baseline execution therefore serves a dual purpose: it confirms protocol viability while also demonstrating that the default delivery path already aligns with an adversary model in which token material can be captured and modified.

4.3 Mechanics of the Signature Bypass Attack (`alg: none`)

The signature bypass attack is executed by transforming a legitimate JWT into a forged token that claims elevated privileges while omitting cryptographic integrity. The procedure begins by decoding the Base64URL-encoded header and payload to expose the claim set. The attacker then mutates authorization-critical claims, most notably changing `"role": "user"` to `"role": "admin"`, while preserving ancillary claims such as issuer, audience, and timestamps to avoid triggering naive validation heuristics. This step exploits the fact that claim values are only trustworthy if the signature is verified; in an unverified context, they are attacker-controlled input.

The core cryptographic exploit consists of rewriting the header field `alg` to the value `none` and then removing the signature segment entirely. Under robust validation, such a token is immediately rejected because the signature is missing and the algorithm is unsupported. However, when a validation routine decodes tokens without verifying signatures or blindly accepts the algorithm value from the header, the forged token is treated as legitimate. The exploit therefore relies on two compounding failures: trusting the user-supplied `alg` value and treating a signatureless token as structurally valid. This bypass converts the integrity of the token from a cryptographic property into a mere syntactic property, allowing privilege escalation through claim manipulation.

4.4 Exploitation Phase and Vulnerable Backend Analysis

The vulnerable FastAPI resource server is intentionally implemented to demonstrate the consequences of unverified token processing. The critical defect is a decoding path that explicitly disables signature verification and other registered claim checks while still consuming claims to authorize access. The excerpt below, taken from the experimental data export, shows a decoding routine that sets `verify_signature` to `False` and then proceeds to authorize requests solely on a `role` claim extracted from the decoded payload. This pattern collapses the authorization boundary into a client-controlled input channel, which is precisely the condition required for the `alg: none` exploit to succeed.

```
1 class VulnerableJwtAuth:
2     """Intentionally vulnerable JWT handler for security testing."""
3
4     def decode_unverified(self, token: str) -> Dict[str, Any]:
5         """Decode a JWT without verifying its signature or claims (vulnerable)."""
6         return jwt.decode(
7             token,
8             options={
9                 "verify_signature": False,
10                "verify_aud": False,
11                "verify_iss": False,
12                "verify_exp": False,
13            },
14            algorithms=self._accepted_algorithms,
15        )
16
17     try:
18         token = self._auth.extract_bearer_token(request.headers.get("Authorization"))
```

```

22         ))
23         # Intentionally insecure: decode without signature verification.
24         claims = self._auth.decode_unverified(token)
25     except Exception:
26         return JSONResponse(status_code=401, content={"detail": "Unauthorized"})
27
28     if claims.get("role") != "admin":
29         return JSONResponse(status_code=403, content={"detail": "Forbidden"})

```

Listing 4.1: Vulnerable JWT decoding and authorization path (signature verification disabled)

The corresponding attack execution log, also captured in the phase data export, demonstrates that the forged token is accepted and that the backend responds with HTTP 200, confirming administrative access. The log includes the manipulated header and payload as well as the response from the vulnerable endpoint.

```

1 FORGED_HEADER: {'alg': 'none', 'typ': 'JWT',
2   'kid': 'abUch0DotE6lgU9Vy-t3da2o15yX8maEDRKC2hiCI9o'}
3 FORGED_PAYLOAD: {'exp': 1780091674, 'iat': 1780091374, 'auth_time': 1780091374, 'jti':
4   '81d666c0-0c66-4d70-922d-91c0cfcde40c',
5   'iss': 'http://localhost:8080/realms/security-lab', 'aud': 'account', 'sub': '63
6   c4f288-e40f-493c-83e2-5ad53ce55e62', 'typ': 'Bearer',
7   'azp': 'secure-client', 'session_state': '3987515d-e6bf-4445-bd99-1c5280925422', 'acr
8   ': '1',
9   'allowed-origins': ['https://secure.example.com'], 'realm_access': {'roles': ['
10   default-roles-security-lab', 'offline_access', 'uma_authorization']},
11   'resource_access': {'account': {'roles': ['manage-account', 'manage-account-links', '
12   view-profile']}}, 'scope': 'openid profile email',
13   'sid': '3987515d-e6bf-4445-bd99-1c5280925422', 'email_verified': True, 'name': 'Lab
14   User', 'preferred_username': 'lab-user',
15   'given_name': 'Lab', 'family_name': 'User', 'email': 'lab-user@example.com', 'role':
16   'admin'}
17 ATTACKER_VULNERABLE_OUTPUT_START
18 Status: 200
19 {"message": "Welcome to the admin dashboard.", "user": {"exp": 1780091674, "iat":
20   1780091374, "auth_time": 1780091374, "jti": "81d666c0-0c66-4d70-922d-91c0cfcde40c",

```

Listing 4.2: Attacker execution output (excerpted) demonstrating forged token acceptance

The failure mode is therefore structural rather than incidental: the validation routine treats the token as a data container rather than a cryptographic assertion. By disabling signature verification and other registered claim checks, the system implicitly trusts attacker-controlled inputs and allows the token header to dictate its own verification policy. In this design, the `alg: none` header becomes a license to bypass integrity checks, and the mutated `role` claim becomes the sole authorization signal. The result is a deterministic privilege escalation path

that succeeds whenever cryptographic enforcement is omitted or when the algorithm field is accepted without a server-side allowlist. This observation underscores the requirement that token verification must be explicit, server-controlled, and algorithm-constrained to the issuer's intended signing scheme.

Chapter 5

Defensive Strategies and Infrastructure Hardening

5.1 Architectural Shift: Mitigating Interception via PKCE

The defensive posture begins with a structural transition from the implicit flow to the authorization code flow augmented with Proof Key for Code Exchange (PKCE). In the implicit flow, access tokens are delivered through the front channel, making them observable to intermediaries and exposing them to interception or browser-based leakage. By contrast, the authorization code flow isolates token delivery to a back-channel exchange between the client and the Identity Provider, and PKCE further binds that exchange to a one-time cryptographic proof. This shift collapses the attack surface created by front-channel token exposure and ensures that the authorization code cannot be redeemed without possession of the correct verifier.

PKCE (S256) introduces a verifiable linkage between the initial authorization request and the later token exchange. The client generates a high-entropy `code_verifier`, hashes it with SHA-256, and encodes the digest using Base64URL to produce the `code_challenge`. The authorization server records the challenge with the issued authorization code, and the token endpoint later verifies that the provided verifier maps to the stored challenge. This mechanism ensures that interception of the authorization code alone is insufficient for token redemption, thereby mitigating replay and redirect capture attacks.

```
1 def generate_code_verifier(length: int = 64) -> str:
2     """Generate a high-entropy code_verifier string."""
3     return secrets.token_urlsafe(length)[:length]
4
5
6 def generate_code_challenge(code_verifier: str) -> str:
7     """Create a Base64 URL-safe SHA-256 code_challenge from the verifier."""
8     digest = hashlib.sha256(code_verifier.encode("ascii")).digest()
9     return base64.urlsafe_b64encode(digest).decode("ascii").rstrip("=")
```

Listing 5.1: PKCE S256 code verifier and challenge generation

5.2 The Hardened Resource Server Implementation

The hardened FastAPI resource server implements strict token validation using the OpenID Connect discovery metadata and JWKS distribution channel. The verification layer relies on `PyJWKClient` to fetch the active signing keys from the Keycloak JWKS endpoint, which is typically advertised at the discovery path and materialized at the `/.well-known/jwks.json` URL. This dynamic retrieval enables key rotation without redeploying resource servers, while preserving strong issuer control over signing material. In the defensive configuration, the resource

server enforces RS256 exclusively and rejects `none` or symmetric alternatives such as HS256 by applying a server-side algorithm allowlist prior to signature verification.

The verification logic also enforces registered claim validation. Issuer (`iss`) and audience (`aud`) are strictly matched against configured values, and expiration (`exp`) is required and verified to prevent acceptance of stale tokens. The following excerpt from the secure server implementation, extracted from the empirical data export, demonstrates explicit algorithm checks, JWKS key acquisition, and claim validation using the PyJWT library.

```
1 class SecureJwtAuth:
2     """JWT validator that enforces RS256 and standard claim checks."""
3
4     def verify_and_decode(self, token: str) -> Dict[str, Any]:
5         """Verify the JWT signature and validate registered claims."""
6         header = jwt.get_unverified_header(token)
7         if header.get("alg") != self._settings.allowed_algorithm:
8             raise InvalidAlgorithmError("Unsupported JWT algorithm.")
9
10        signing_key = self._jwks_client.get_signing_key_from_jwt(token)
11        return jwt.decode(
12            token,
13            key=signing_key.key,
14            algorithms=[self._settings.allowed_algorithm],
15            issuer=self._settings.issuer,
16            audience=self._settings.audience,
17            options={
18                "require": ["exp", "iss", "aud"],
19                "verify_signature": True,
20                "verify_exp": True,
21                "verify_aud": True,
22                "verify_iss": True,
23            },
24        )
25
26        try:
27            token = self._auth.extract_bearer_token(request.headers.get("Authorization"))
28            claims = self._auth.verify_and_decode(token)
29        except ExpiredSignatureError:
30            return JSONResponse(status_code=401, content={"detail": "Token expired."})
31        except (InvalidSignatureError, InvalidAlgorithmError):
32            return JSONResponse(status_code=401, content={"detail": "Invalid token signature."})
```

Listing 5.2: Secure JWT validation with JWKS and RS256 enforcement

5.3 Session Lifespan Constriction and Refresh Token Rotation (RTR)

Token lifespan constriction is a critical element of defensive hardening because bearer tokens are transferable credentials whose compromise cannot be revoked without explicit server-side controls. The secure configuration reduces access token lifetime to a short window, approximately five minutes, which sharply limits the operational value of any leaked token. This approach transforms token theft from a persistent risk into a transient event, thereby improving the efficacy of detection and incident response. Short lifetimes also reduce the probability that compromised tokens remain valid long enough to be replayed across multiple service boundaries.

Refresh Token Rotation (RTR) further strengthens session integrity by ensuring that each

refresh token is single-use. Upon refresh, the previously issued token is invalidated and replaced with a new value, and any attempt to reuse the prior token is treated as an indicator of compromise. This mechanism is particularly important in distributed environments where refresh tokens may transit through multiple clients or intermediaries. RTR enforces a monotonic chain of refresh operations and surfaces replay attempts as anomalous events, enabling the identity provider to revoke sessions proactively and prevent silent token reuse.

5.4 Mitigation Proof: Exploit Rejection and Validation

The effectiveness of the defensive controls is validated by replaying the exact `alg: none` token forgery attempt against the hardened resource server. The attacker script submits a forged token identical in structure to the vulnerable scenario, yet the secure server rejects it due to signature and algorithm enforcement. The raw execution log below demonstrates that the request is blocked with HTTP 401 and a signature validation error response, providing empirical confirmation that the mitigation path is effective.

```
1 ATTACKER_SECURE_OUTPUT_START
2 Status: 401
3 {"detail":"Invalid token signature."}
4
5 ATTACKER_SECURE_OUTPUT_END
```

Listing 5.3: Attacker output against hardened server showing rejection

This outcome validates the defensive architecture on two levels: cryptographic enforcement prevents acceptance of unsigned tokens, and strict algorithm filtering ensures that the server does not defer to attacker-supplied header semantics. In combination with short-lived tokens and refresh rotation, the hardened system restores the integrity boundary between issuer and resource server and converts previously exploitable paths into controlled, observable rejection events.

Chapter 6

Automated Integration and Empirical Verification Test Results

6.1 Automated Security Testing Methodology

The final QA validation phase is implemented using the Python `pytest` automation framework, which enables deterministic, repeatable security assertions across isolated service configurations. The test suite is designed to encode the security posture of both vulnerable and hardened systems as explicit, programmatic expectations. In the vulnerable configuration, the suite validates that a forged token with `alg: none` and an elevated `role` claim is accepted and yields HTTP 200, thereby confirming that the attack simulation is operational and measurable. In the secure configuration, the same forged token is expected to be rejected with HTTP 401, which verifies that signature enforcement, algorithm allowlisting, and claim validation are consistently active.

This dual assertion strategy is not a mere functional test; it is a security regression guardrail. It ensures that the vulnerability demonstration remains reproducible for research integrity while simultaneously proving that the hardened path enforces cryptographic verification under identical request semantics. By encoding these checks at the integration layer, the test suite captures real HTTP behavior across the complete IdP-client-resource-server chain rather than relying on unit-level assumptions about cryptographic correctness.

6.2 Empirical Test Execution and Log Analysis

The following terminal output shows the full execution of the `pytest` suite as captured in the raw data export. The logs confirm that all tests pass, including the deliberate acceptance of the forged token in the vulnerable path and the enforced rejection in the hardened path. This 100% pass rate demonstrates both the correctness of the exploit simulation and the effectiveness of the defensive controls.

```
1 ===== test session starts =====
2 platform win32 -- Python 3.12.4, pytest-8.2.2, pluggy-1.6.0 -- E:\University\Semester
   8\Network Security\Project\DistributedAuthSecurityAnalysis\.venv\Scripts\python.
   exe
3 cachedir: .pytest_cache
4 rootdir: E:\University\Semester 8\Network Security\Project\
   DistributedAuthSecurityAnalysis
5 plugins: anyio-4.13.0
6 collected 3 items
7
8 tests/test_security_flows.py::TestOidcSecurity::test_vulnerable_accepts_forged_token
   PASSED [ 33%]
```

```

9 tests/test_security_flows.py::TestOidcSecurity::test_secure_rejects_forged_token
  PASSED [ 66%]
10 tests/test_security_flows.py::TestOidcSecurity::test_secure_rejects_expired_token
  PASSED [100%]
11
12 ===== 3 passed in 12.43s =====

```

Listing 6.1: Pytest execution output for security flow validation

6.3 Significance of Security Regressions in DevSecOps

Automated integration tests represent a foundational control in DevSecOps pipelines because they convert security assumptions into enforceable, continuously evaluated constraints. When embedded into CI/CD workflows, these tests provide immediate feedback whenever a change modifies token validation behavior, such as removing JWKS checks or relaxing algorithm restrictions. The result is a self-policing security boundary that is enforced by the build system rather than by manual review alone.

From an architectural perspective, this approach prevents security drift in distributed environments where multiple teams contribute to identity and authorization logic. By ensuring that forged tokens are rejected under the hardened path and accepted only within explicitly vulnerable test environments, the QA framework guarantees that future development cannot inadvertently reintroduce the same validation flaws. In effect, automated security regression testing becomes a permanent guardrail that preserves cryptographic integrity and reinforces a disciplined, policy-driven approach to token verification.

Chapter 7

Conclusion and Security Recommendations

7.1 Conclusion and Key Findings

This study demonstrates that distributed authentication and authorization systems are only as strong as their most permissive validation boundary. The empirical findings show that misconfigurations in identity issuance and token verification can transform otherwise standardized OAuth 2.0 and OIDC flows into bypassable trust channels, particularly when validation is reduced to syntactic parsing rather than cryptographic verification. The intentional weakening of realm and client settings, coupled with a vulnerable token decoding path, revealed how front-channel exposure, excessive token lifetimes, and permissive redirect targets amplify the consequences of weak validation logic. Conversely, the hardened architecture restored the integrity of the authorization perimeter by enforcing asymmetric cryptographic verification, strict claim validation, and defensible token lifecycles.

The most critical lesson is the catastrophic nature of programmatic validation flaws such as the `alg: none` bypass. When a resource server accepts header-directed algorithms or disables signature checks, it forfeits the integrity guarantees that are foundational to token-based security. In such cases, the attacker becomes the issuer, and any authorization decision derived from token claims is reduced to attacker-controlled input. The secure configuration demonstrates that enforcing asymmetric verification, anchored in centralized IdPs and JWKS-based key distribution, is not optional but essential for maintaining consistent authorization semantics in distributed environments.

7.2 Security Recommendations for Cloud-Native Architectures

Cloud-native microservices should adopt a security baseline that assumes hostile networks and untrusted client contexts. Public clients must use PKCE with the authorization code flow to prevent interception of authorization codes and eliminate front-channel token leakage. Resource servers must validate tokens against a strict allowlist of algorithms (for example, `RS256`) and must reject unsigned tokens and symmetric alternatives that are incompatible with the issuer's signing policy. JWKS-based key retrieval should be mandatory, with issuer and audience validation enforced as first-class requirements rather than optional checks.

Operationally, refresh token rotation and short-lived access tokens should be standard practice to reduce replay windows and provide a measurable signal for compromise. Security regression tests that assert both acceptance and rejection paths must be integrated into CI/CD pipelines to prevent accidental erosion of validation logic. By combining PKCE, strict JWKS validation, RTR, and automated regression tests, organizations can maintain robust authorization perimeters and prevent future reintroduction of token handling vulnerabilities.

Bibliography

- [1] IEEE, “[IEEE Conference Paper 10.1109 usbereit58508.2023.10158865 uxae](#),” 2023.
- [2] NCS, “[Labtainers Framework \(17NCS\)](#),” Accessed 2026.
- [3] Document, “[Document 25882618](#),” Accessed 2026.
- [4] Document, “[ID5662 1834-1852](#),” Accessed 2026.
- [5] OWASP, “[OWASP Top 10 \(2021\) - A01: Broken Access Control](#),” 2021.
- [6] MDPI, “[Electronics 12-00940](#),” 2023.
- [7] Auth0, “[Exploiting JWT Vulnerabilities](#),” Accessed 2026.
- [8] OpenID Foundation, “[OpenID Connect Core 1.0 \(Errata Set 2\)](#),” 2014.
- [9] Document, “[Free Chapter v1.0.0](#),” Accessed 2026.
- [10] J. Vrancken, “[A Methodology for Penetration Testing Docker Systems](#),” Accessed 2026.
- [11] IETF, “[JSON Web Token Claims](#),” Accessed 2026.
- [12] Okta Developer, “[OAuth 2.0 and OpenID Connect Overview](#),” Accessed 2026.
- [13] Okta, “[Okta Platform Release Overview \(Q2\)](#),” Accessed 2026.
- [14] IETF, “[RFC 6749 - The OAuth 2.0 Authorization Framework](#),” 2012.
- [15] IETF, “[RFC 7517 - JSON Web Key \(JWK\)](#),” 2015.
- [16] IETF, “[RFC 7519 - JSON Web Token \(JWT\)](#),” 2015.
- [17] IETF, “[RFC 7636 - Proof Key for Code Exchange \(PKCE\)](#),” 2015.
- [18] IETF, “[RFC 7662 - OAuth 2.0 Token Introspection](#),” 2015.
- [19] IETF, “[RFC 8252 - OAuth 2.0 for Native Apps](#),” 2017.
- [20] IETF, “[RFC 8705 - OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Tokens](#),” 2020.
- [21] IETF, “[RFC 9449 - OAuth 2.0 Demonstrating Proof of Possession \(DPoP\)](#),” 2023.
- [22] IETF, “[RFC 9700 - OAuth 2.0 Security Best Current Practice](#),” 2024.
- [23] Project Documentation, “[Phase 1 - Theoretical Foundations of Modern Authentication and Authorization Protocols](#),” 2026.
- [24] Project Documentation, “[Phase 2 - Laboratory Environment Design and Tool Specification](#),” 2026.

- [25] Project Documentation, “[Phase 3 - Scenario Implementation and Attack Simulation](#),” 2026.
- [26] Project Documentation, “[Phase 4 - Defensive Strategies and Infrastructure Hardening](#),” 2026.